LCE

Manual del lenguaje y guía de referencia

Aprende a crear aplicaciones nativas con Qt
desde cero y en español

Versión 0.1.0 · Noviembre de 2026
Sintaxis en español · Compilación a C++/Qt6

Índice

1. ¿Qué es LCE? 4

1.1. ¿Para quién es este manual? 4

1.2. Cómo funciona 4

2. Instalación 5

2.1. Instalar el compilador (.deb) 5

2.2. Instalar la extensión de VS Code 5

2.3. Compilar el motor desde el código fuente (opcional) 5

3. Tu primer programa 6

3.1. ¿Qué hace cada línea? 6

4. Fundamentos del lenguaje 7

4.1. Estructura de un programa 7

4.2. Comentarios 7

4.3. Variables y tipos 7

4.4. Asignación 7

4.5. Operadores 8

5. Control de flujo 9

5.1. Condicional si / sino / sino si 9

5.2. Bucle mientras 9

5.3. Bucle para 9

6. Funciones 10

6.1. Funciones sin retorno 10

6.2. Parámetros por referencia 10

6.3. Prototipos y orden 10

7. Interfaz gráfica con Qt 11

7.1. Crear ventana y widgets 11

7.2. Colocar y mostrar 11

7.3. Eventos 11

7.4. Leer y escribir en una caja de texto 11

7.5. Ejemplo completo: formulario 11

8. Referencia de palabras clave 12

9. Referencia de funciones nativas 12

10. El compilador lce 13

10.1. Flujo típico 13

10.2. Depurar la generación 13

10.3. Mensajes de error 13

11. VS Code: atajos y productividad 14

11.1. Fragmentos disponibles 14

11.2. Tarea de compilación 14

12. Solución de problemas 15

13. Glosario 16

Apéndice A. Gramática (EBNF) 17

Apéndice B. Chuleta rápida 18

Apéndice C. Recursos 18

1. ¿Qué es LCE?

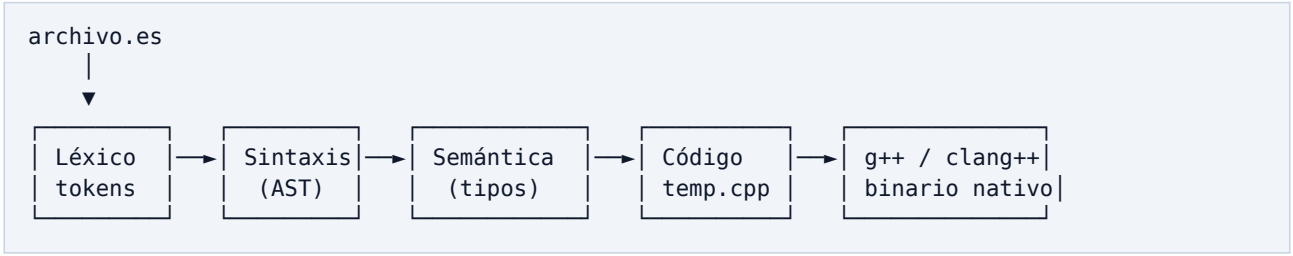
LCE es un lenguaje de programación con **sintaxis 100 % en español** diseñado para crear aplicaciones de escritorio nativas con el framework **Qt (C++)**. El código se escribe en archivos `.es` (y también `.lce`), se traduce a C++17/Qt6 y se compila a un **binario nativo de alto rendimiento**.

No es un intérprete ni una máquina virtual: LCE es un **transpilador**. El compilador `lce` lee tu programa, comprueba que los tipos y las declaraciones sean correctos y genera C++ que `g++` (o `clang++`) convierte en un ejecutable que corre directamente sobre el sistema operativo.

1.1. ¿Para quién es este manual?

Para **programadores** que quieran una sintaxis clara en español y la potencia de Qt, y para **personas que nunca han programado** que quieran aprender desde cero con ejemplos guiados. No necesitas saber C++ para empezar, aunque ayuda.

1.2. Cómo funciona



Cada fase produce errores claros **en español**. Si algo falla, recibirás un mensaje del tipo `Error semántico: La variable 'x' no está declarada` antes de llegar al compilador de C++.

> **Requisitos**
> Linux (probado en Ubuntu/Debian), `g++` o `clang++`, `pkg-config` y las bibliotecas de desarrollo de Qt6 (`qt6-base-dev`). El compilador `lce` y la extensión de VS Code se instalan en los siguientes apartados.

2. Instalación

2.1. Instalar el compilador (.deb)

Descarga `lce_0.1.0_amd64.deb` y ejecuta:

```
sudo apt install ./lce_0.1.0_amd64.deb
# alternativa:
sudo dpkg -i lce_0.1.0_amd64.deb
```

El binario queda en `/usr/bin/lce`. Comprueba la instalación:

```
which lce          # -> /usr/bin/lce
lce --help
```

2.2. Instalar la extensión de VS Code

Instala el paquete `.vsix` desde la terminal:

```
code --install-extension lce-0.1.0.vsix
```

O desde la interfaz: **Extensiones → menú ⋯ → Instalar desde VSIX**. La extensión aporta resaltado de sintaxis, fragmentos (snippets), comentarios, autocompletado de llaves y una tarea para compilar con **Ctrl+Shift+B**.

Si usas OpenCode o prefieres una instalación manual, descomprime `lce-0.1.0.zip` en `~/.vscode/extensions/lce/`.

2.3. Compilar el motor desde el código fuente (opcional)

Si prefieres compilar tú mismo el compilador, necesitas Rust:

```
cd motor
cargo build --release      # binario en target/release/lce
cargo test                 # 90 tests
```

> **Consejo**
> Para que la tarea de VS Code encuentre el compilador, `lce` debe estar en el PATH. El paquete `.deb` ya lo deja en `/usr/bin/lce`.

3. Tu primer programa

Crea un archivo llamado `hola.es` con este contenido:

```
programa Hola {

    ventana v = crear_ventana("Mi primera app", 400, 300)
    boton b = crear_boton("Haz clic aquí")

    al hacer_clic(b) {
        mostrar_mensaje("Información", "¡Hola desde LCE!")
    }

    agregar(v, b)
    mostrar(v)
}
```

Compila y ejecuta:

```
lce hola.es -o hola
./hola
```

Se abrirá una ventana con un botón; al pulsarlo aparece un cuadro de diálogo. Enhorabuena: acabas de crear una aplicación nativa de Qt.

3.1. ¿Qué hace cada línea?

- `programa Hola { ... }` define el programa y su nombre. Todo lo que está dentro se ejecuta al arrancar.
- `ventana v = crear_ventana(...)` crea una ventana principal (`QMainWindow`) con título y tamaño.
- `boton b = crear_boton(...)` crea un botón (`QPushButton`).
- `al hacer_clic(b) { ... }` registra el código que se ejecuta al pulsar el botón.
- `agregar(v, b)` coloca el botón dentro de la ventana; `mostrar(v)` muestra la ventana.

Sin punto y coma

En LCE cada instrucción termina con un **salto de línea**. El punto y coma `;` es opcional. Las líneas en blanco se ignoran.

4. Fundamentos del lenguaje

4.1. Estructura de un programa

Todo programa empieza con la palabra `programa`, un nombre y un bloque entre llaves. Dentro del bloque pueden aparecer variables, control de flujo, funciones y llamadas a funciones nativas de Qt.

4.2. Comentarios

Los comentarios se escriben con `//` o con `#` y llegan hasta el final de la línea:

```
// Esto es un comentario
# Esto también
entero edad = 25   // comentario al final de la línea
```

4.3. Variables y tipos

Una variable se declara con su **tipo**, un nombre y un valor inicial:

```
entero   edad     = 25
decimal  precio   = 19.99
booleano activo   = verdadero
cadena   nombre   = "Ana"
```

Tipo	Equivalente en C++	Qué guarda
entero	int	Números enteros (1, -7, 42)
decimal	double	Números con decimales (3.14)
booleano	bool	verdadero o falso
cadena	QString	Texto entre comillas
nada	void	Ausencia de valor (solo en funciones)
ventana	QMainWindow*	Una ventana principal
boton	QPushButton*	Un botón
etiqueta	QLabel*	Una etiqueta de texto
caja_texto	QLineEdit*	Un campo de texto

4.4. Asignación

Para cambiar el valor de una variable ya declarada se usa `=`:

```
entero contador = 0
contador = contador + 1
```

4.5. Operadores

Categoría	Operadores	Ejemplo
Aritméticos	+ - * / %	a + b * 2
Comparación	== != < > <= >=	edad >= 18
Lógicos	y o no	activo y no bloqueado
Concatenación	+	"Hola, " + nombre
Negación	- no	-5 no verdadero

Los operadores tienen la precedencia habitual: primero *, /, %; después +, -; luego las comparaciones; y por último y / o. Usa paréntesis para dejar clara la intención:

```
booleano mayor = (edad >= 18) y (saldo > 0)
```

Concatenar texto

El operador + suma números y también **une cadenas**: `"Hola, " + nombre` produce un único texto.

5. Control de flujo

5.1. Condicional si / sino / sino si

La condición debe ser de tipo `booleano`:

```
si (edad >= 18) {
    mostrar_mensaje("Acceso", "Bienvenido")
} sino si (edad >= 13) {
    mostrar_mensaje("Acceso", "Acceso juvenil")
} sino {
    mostrar_mensaje("Acceso", "Acceso denegado")
}
```

`sino si` permite encadenar condiciones; se traduce a `else if` y admite tantas ramas como necesites.

5.2. Bucle mientras

Repite un bloque mientras la condición sea verdadera:

```
entero contador = 0
mientras (contador < 3) {
    contador = contador + 1
}
```

5.3. Bucle para

Recorre un rango de enteros. La variable se crea automáticamente y el límite `hasta` es **exclusivo**:

```
entero suma = 0
para i desde 1 hasta 6 {
    suma = suma + i     // i = 1, 2, 3, 4, 5
}
```

`para i desde 1 hasta 6` se traduce a `for (int i = 1; i < 6; i++)`.

6. Funciones

Una función agrupa código reutilizable. Se declara con `funcion`, un nombre, parámetros con tipo y, opcionalmente, un tipo de retorno con `->`:

```
funcion suma(entero a, entero b) -> entero {
    retornar a + b
}

entero total = suma(3, 4)     // total = 7
```

6.1. Funciones sin retorno

Si se omite el tipo de retorno, la función devuelve nada (void):

```
funcion saludar(cadena nombre) {
    mostrar_mensaje("Bienvenida", nombre)
}
```

6.2. Parámetros por referencia

Con & el parámetro se pasa por referencia y los cambios afectan a la variable original del llamador:

```
funcion incrementar(entero &valor) {
    valor = valor + 1
}

entero contador = 0
incrementar(contador)     // contador = 1
```

Referencias

El argumento de un parámetro & debe ser una **variable** (no un literal) y del mismo tipo. `incrementar(5)` produce un error semántico.

6.3. Prototipos y orden

Puedes llamar a una función definida más abajo en el archivo: el compilador emite primero los prototipos de todas las funciones y luego sus definiciones. Esto permite también la **recursión**:

```
funcion factorial(entero n) -> entero {
    si (n <= 1) {
        retornar 1
    }
    retornar n * factorial(n - 1)
}
```

7. Interfaz gráfica con Qt

LCE incluye funciones nativas para construir interfaces. Cada widget se guarda en una variable de su tipo y luego se coloca en la ventana con `agregar`.

7.1. Crear ventana y widgets

```
ventana v = crear_ventana("Título", 500, 350)
boton b = crear_boton("Aceptar")
etiqueta e = crear_etiqueta("Hola")
caja_texto c = crear_caja_texto("Escribe aquí")
```

7.2. Colocar y mostrar

```
agregar(v, e)      // añade la etiqueta al layout vertical
agregar(v, c)      // añade la caja de texto
agregar(v, b)      // añade el botón
mostrar(v)         // muestra la ventana
```

Los widgets se apilan en un layout vertical en el orden en que se agregan.

7.3. Eventos

```
al hacer_clic(b) {
    mostrar_mensaje("Título", "Texto del mensaje")
}
```

7.4. Leer y escribir en una caja de texto

```
cadena texto_usuario = texto(c)        // lee el contenido
establecer_texto(c, "Nuevo valor")     // cambia el contenido
```

7.5. Ejemplo completo: formulario

```
programa Formulario {

    ventana v = crear_ventana("Saludo", 420, 220)
    caja_texto nombre = crear_caja_texto("Escribe tu nombre")
    boton b = crear_boton("Saludar")

    al hacer_clic(b) {
        cadena n = texto(nombre)
        mostrar_mensaje("Saludo", "Hola, " + n)
    }

    agregar(v, nombre)
    agregar(v, b)
    mostrar(v)
}
```

8. Referencia de palabras clave

Palabra	Equivalente	Uso
programa	main	Inicia el programa
funcion	function	Define una función
si	if	Condición
sino	else	Alternativa
mientras	while	Bucle condicional
para	for	Bucle con rango
desde / hasta	inicio / fin	Límites del bucle para
retornar	return	Devuelve un valor
entero	int	Tipo entero
decimal	double	Tipo decimal
booleano	bool	Tipo lógico
cadena	QString	Tipo texto
nada	void	Sin valor
ventana	QMainWindow*	Ventana principal
boton	QPushButton*	Botón
etiqueta	QLabel*	Etiqueta de texto
caja_texto	QLineEdit*	Campo de texto
verdadero / falso	true / false	Literales booleanos
al hacer_clic	connect(clicked)	Evento de clic
y / o / no	&& / || / !	Operadores lógicos

9. Referencia de funciones nativas

Función	Firma	Descripción
crear_ventana	(cadena, entero, entero) -> ventana	Crea la ventana principal
crear_boton	(cadena) -> boton	Crea un botón
crear_etiqueta	(cadena) -> etiqueta	Crea una etiqueta
crear_caja_texto	(cadena) -> caja_texto	Crea un campo de texto
mostrar_mensaje	(cadena, cadena) -> nada	Muestra un cuadro de diálogo
agregar	(ventana, widget) -> nada	Añade un widget a la ventana
mostrar	(widget) -> nada	Muestra una ventana o widget
texto	(caja_texto) -> cadena	Lee el texto de la caja
establecer_texto	(caja_texto, cadena) -> nada	Cambia el texto de la caja

10. El compilador lce

Uso general:

```
lce <archivo.es> [opciones]
```

Opción	Descripción
-o <salida>	Nombre del binario resultante (por defecto: programa)
--no-compilar	Solo genera temp.cpp, sin invocar a g++/clang++
--solo-verificar	Analiza léxico, sintaxis y tipos sin generar código
-h, --help	Muestra la ayuda

10.1. Flujo típico

```
lce programa.es -o programa     # verifica, genera C++ y compila
./programa                      # ejecuta el binario nativo
```

10.2. Depurar la generación

```
lce programa.es --solo-verificar   # solo comprueba tipos
lce programa.es --no-compilar      # genera temp.cpp para inspeccionarlo
```

10.3. Mensajes de error

Los errores indican la fase en la que se produjeron:

```
Error léxico: Cadena de texto sin cerrar
Error de sintaxis: Se esperaba ')' (se encontró LlaveDer)
Error semántico: La variable 'z' no está declarada
Error semántico: No se puede asignar un valor de tipo 'cadena'
                 a la variable 'x' de tipo 'entero'
```

11. VS Code: atajos y productividad

La extensión de LCE integra el lenguaje en VS Code con resaltado, fragmentos y compilación. Estos son los atajos más útiles:

Acción	Atajo
Compilar el archivo actual	Ctrl+Shift+B
Ejecutar / depurar	F5
Comentar / descomentar línea	Ctrl+/
Insertar fragmento (snippet)	prefijo + Tab
Paleta de comandos	Ctrl+Shift+P
Buscar en el archivo	Ctrl+F
Abrir terminal integrada	Ctrl+ñ (o Ctrl+`)

11.1. Fragmentos disponibles

Prefijo	Qué inserta
programa	Bloque principal del programa
ventana	Ventana principal
boton	Botón
clic	Manejador al hacer_clic
si	Condicional si / sino
mientras	Bucle mientras
para	Bucle para con rango
mensaje	Cuadro de mensaje

11.2. Tarea de compilación

El archivo `.vscode/tasks.json` de la extensión define la tarea por defecto (Ctrl+Shift+B):

```
lce ${file} -o ${fileDirname}/${fileBasenameNoExtension}
```

También incluye una tarea para generar solo `temp.cpp` y otra para depurar con gdb (F5).

12. Solución de problemas

Síntoma	Causa probable	Solución
lce: command not found	No está instalado	sudo apt install ./lce_0.1.0_amd64.deb
No se pudieron obtener las banderas de Qt6	Falta Qt6	sudo apt install qt6-base-dev
g++: command not found	Falta compilador	sudo apt install build-essential
Error semántico: variable no declarada	Nombre mal escrito	Revisa la ortografía y el ámbito
Error de sintaxis: se esperaba ')'	Paréntesis sin cerrar	Revisa las llaves y paréntesis
Error: la función espera N argumentos	Aridad incorrecta	Pasa el número correcto de argumentos
El botón no responde	Falta al hacer_clic	Asocia el evento al botón

Consejo de depuración

Usa `lce archivo.es --solo-verificar` para detectar errores de tipo antes de compilar. Los mensajes indican exactamente qué variable o argumento falla.

13. Glosario

Término	Significado
Transpilador	Programa que traduce de un lenguaje a otro (LCE → C++).
Token	Unidad mínima del código (palabra, número, símbolo).
AST	Árbol de sintaxis abstracta: representación del programa.
Semántica	Fase que comprueba tipos, declaraciones y ámbitos.
Widget	Elemento de interfaz gráfica (botón, caja, ventana).
Layout	Disposición que organiza los widgets dentro de la ventana.
Binario nativo	Ejecutable que corre directamente en la CPU.

Apéndice A. Gramática (EBNF)

```
programa      := "programa" IDENT "{" { declaracion } "}"

declaracion   := declaracion_variable
               | asignacion
               | "si" "(" expr ")" bloque
                   { "sino" ( bloque | "si" "(" expr ")" bloque ) }
               | "mientras" "(" expr ")" bloque
               | "para" IDENT "desde" expr "hasta" expr bloque
               | "retornar" [ expr ] term
               | "al" "hacer_clic" "(" IDENT ")" bloque
               | llamada term
               | "funcion" IDENT "(" [ parametro { "," parametro } ] ")"
                   [ "->" tipo_retorno ] bloque

parametro     := tipo [ "&" ] IDENT
tipo_retorno  := tipo | "nada"
tipo          := "entero" | "decimal" | "booleano" | "cadena"
               | "ventana" | "boton" | "etiqueta" | "caja_texto"

bloque        := "{" { declaracion } "}"
term          := ";" | salto_de_linea
asignacion    := IDENT "=" expr term

expr          := logico_o
logico_o      := logico_y { "o" logico_y }
logico_y      := igualdad { "y" igualdad }
igualdad      := comparacion { ("==" | "!=") comparacion }
comparacion   := termino { ("<" | ">" | "<=" | ">=") termino }
termino       := factor { ("+" | "-") factor }
factor        := unario { ("*" | "/" | "%") unario }
unario        := ("-" | "no") unario | primario
primario      := NUMERO | CADENA | "verdadero" | "falso"
               | IDENT [ "(" [ expr { "," expr } ] ")" ]
               | "(" expr ")"
```

Apéndice B. Chuleta rápida

```
// Programa mínimo
programa P {
    ventana v = crear_ventana("App", 400, 300)
    boton b = crear_boton("OK")
    al hacer_clic(b) { mostrar_mensaje("Hola", "Mundo") }
    agregar(v, b)
    mostrar(v)
}

// Variables
entero x = 10
decimal d = 3.14
booleano ok = verdadero
cadena s = "texto"

// Control
si (x > 0) { ... } sino { ... }
mientras (x > 0) { x = x - 1 }
para i desde 0 hasta 10 { ... }

// Funciones
funcion f(entero a, entero &b) -> entero { retornar a + b }
```

Apéndice C. Recursos

- Código fuente del motor: carpeta `motor/` (Rust).
- Ejemplos completos: carpeta `ejemplos/` (6 programas).
- Extensión de VS Code: carpeta `extension/` y paquete `lce-0.1.0.vsix`.
- Paquete de instalación: `lce_0.1.0_amd64.deb`.

LCE · Manual del lenguaje · Versión 0.1.0 · Noviembre de 2026